

DIGITAL ELECTRONICS

ET 8301

Friday, December 13, 2024

Mr. Cuthbert J. Karawa

Lecture 06

Functions of Combinational Logic

College of Information and Communication Technology (CoICT)

Department Of Electronics And Telecommunications Engineering.

Introduction

- The tradition way of *combinational circuit* design involves simplification and realization of circuit using logic gates.
- This method is effective for *small circuit*.
- When the circuit is more complex, more number of gates are required with more number of wires between them.
- Designing such circuits is time consuming and less reliable.

Introduction

- To avoid these problems several combination circuits available in integrated circuits are employed in the designing of digital systems.

Number of gates per package	Classification	Abbreviation
<10	Small Scale Integration	SSI
10-99	Medium Scale Integration	MSI
1000	Large Scale Integration	LSI
10K	Very Large Scale Integration	VLSI

Introduction

- Most combinational circuits which available in IC are classified as MSI components
- MSI components which available in IC includes *adders, subtractors, multiplexers, demultiplexers, decoders, encoders* and *comparators*.
- These MSI are used to implement combinational circuits.

Binary adder

- Adder are important in computer and also in other types of digital system in which numerical data are processed.
- The basic operations are the arithmetic operations which includes additions of binary digits.

Binary adder

Half adder

- Basic rules of binary addition are performed by a *half adder*, which has two binary inputs (*A* and *B*) and two binary outputs (*Carry out* and *Sum*).
- The inputs and outputs can be summarized on a truth table.

<i>A</i>	<i>B</i>	<i>C_{out}</i>	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Σ = sum

C_{out} = output carry

A and *B* = input variables (operands)

Binary adder

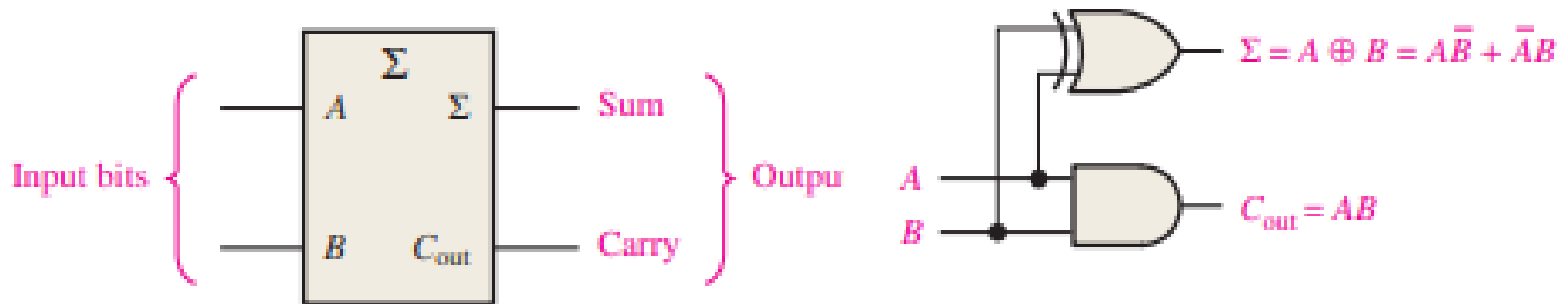
➤ Half Adder

➤ From truth table we get

$$\text{Sum} = A \oplus B \text{ and}$$

$$C_{out} = AB$$

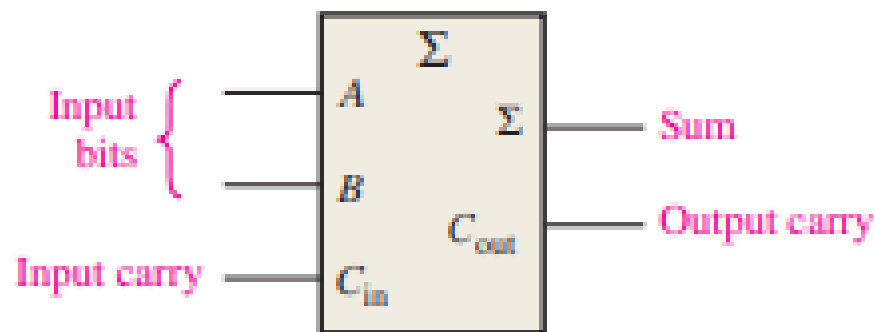
➤ The logic symbol and equivalent circuit are:



Binary adder

Full Adder

- The full adder accepts **two inputs bits** and **input carry** and generates a sum output and an output carry.
- The truth table summarizes the operation.



A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

C_{in} = input carry, sometimes designated as CI

C_{out} = output carry, sometimes designated as CO

Σ = sum

A and B = input variables (operands)

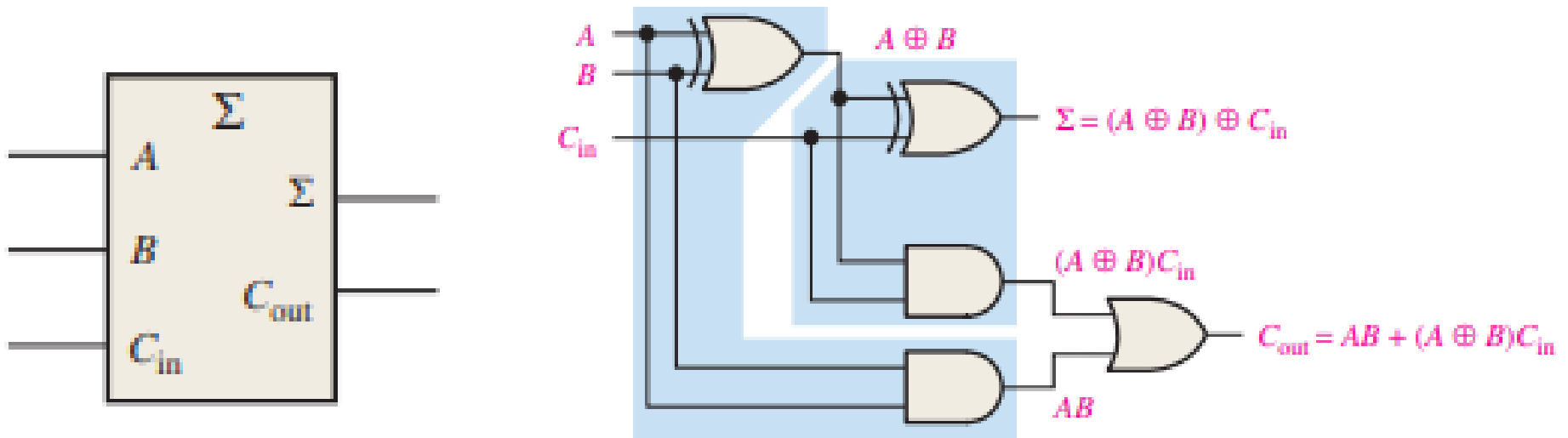
Binary adder

➤ From the truth table we get

$$\text{Sum} = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + BC_{in} + AC_{in}$$

➤ The logic symbol and equivalent circuit are:



Exercise

- Implement full adder by using two half adders.

Binary adder

Parallel Adder

- More often there a need to add *two n bits* numbers.
- When adding two *n bit* number the two LSB are added to produce their sum and carry out bit.
- Then the next two bits are added together with a carry generated from the addition of the first two bits to produce the sum and carry out bits
- The process is repeated until the most significant bits are reached.

Binary adder

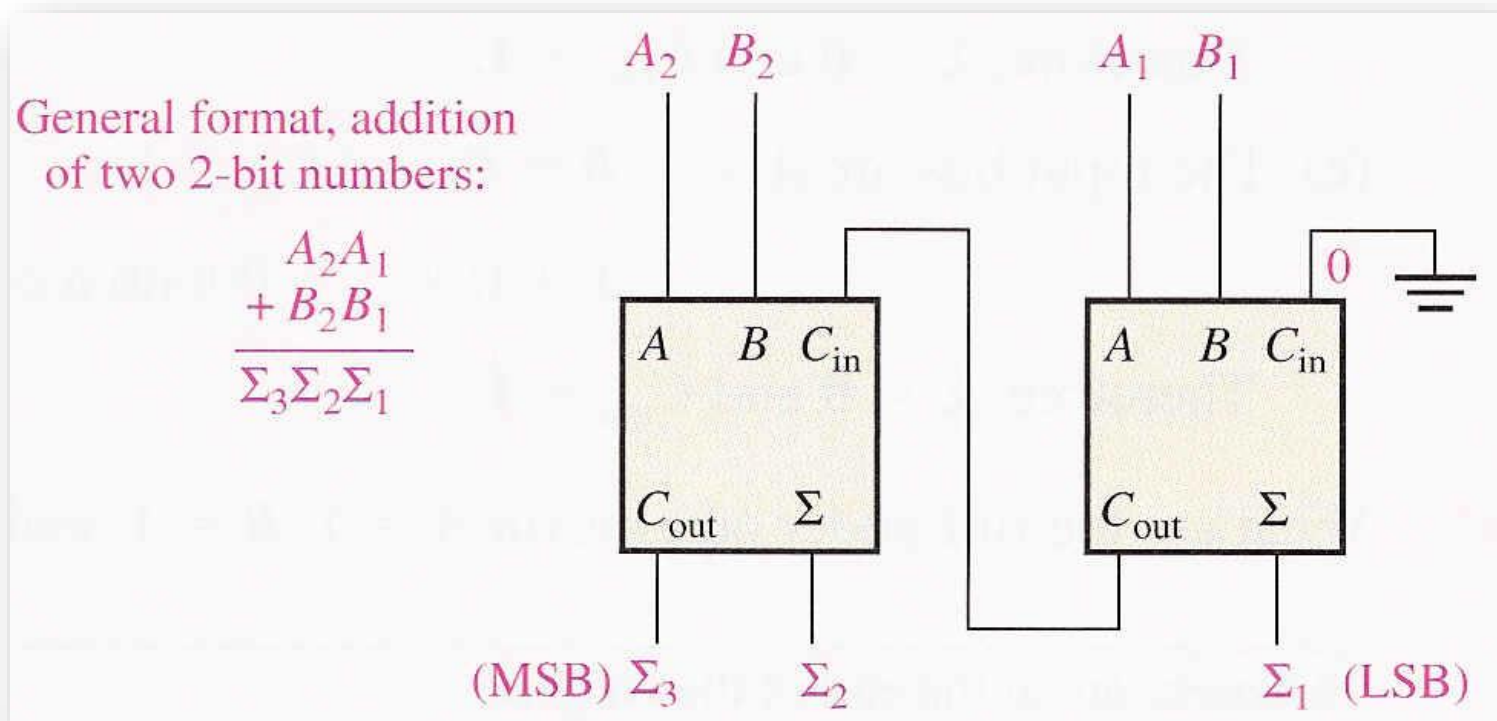
Parallel Adder

- To add two binary numbers a full adder is required for each bit in the numbers.
- Therefore for 2-bit numbers two adders are needed for 3-bit numbers three adders are needed and so on
- The carry output of each adder is connected to the carry input of the next higher order adder.
- Two or more full-adders are connected to form parallel binary adders.

Binary adder

Parallel Adder

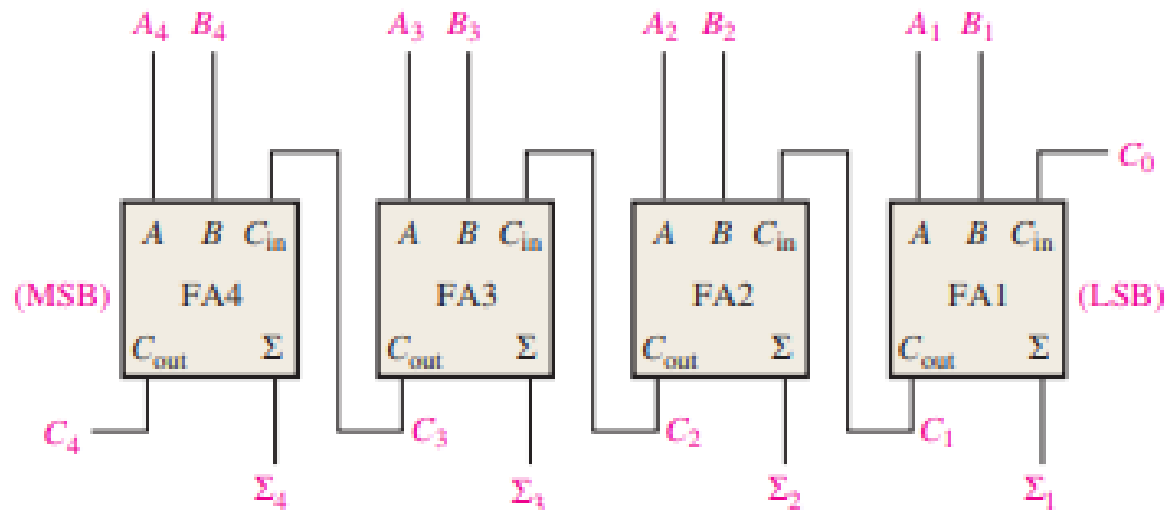
- Figure below shows a basic 2-bit parallel adder which is implemented with two full adder stages.



Binary adder

Parallel Adders

➤ A 4-bit adder is shown figure below.



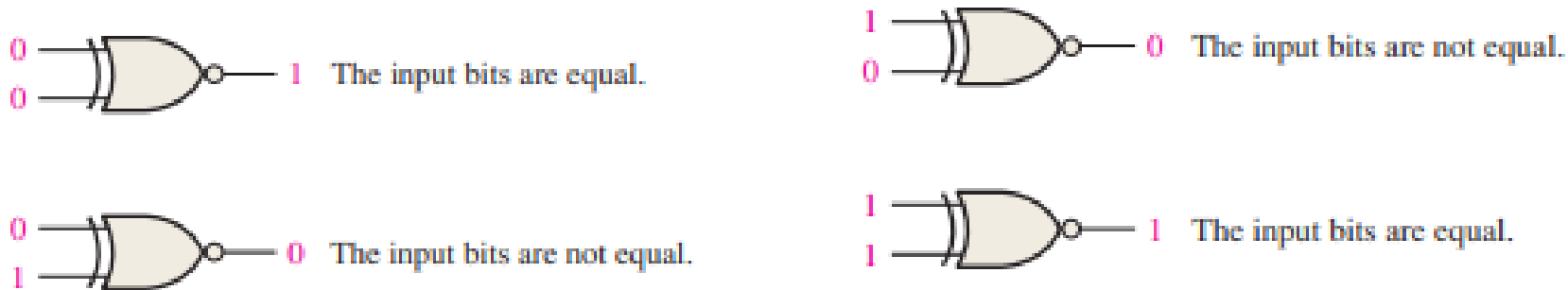
➤ The output carry (C_4) is not ready until it propagates through all of the full adders. This is called ripple carry, delaying the addition process.

Exercise

- Determine the sum generated by the 3-bit parallel adder and show the intermediate carries when the binary number 101 and 011 are being added

Comparators

- The basic function of a **comparator** is to compare the magnitudes of **two binary quantities** to **determine the relationship of those quantities**.



- The exclusive-NOR gate can be used as a basic comparator because its output is a 0 if the two input bits are not equal and a 1 if the input bits are equal.
- Figure above shows the exclusive-NOR gate as a 2-bit comparator.

Decoders

- A **decoder** is a digital circuit that detects the presence of a specified combination of bits (code) on its inputs and indicates the presence of that code by a specified output level.
- In general a decoder has **multiple inputs** and **multiple outputs**.
- There are several types of decoder but the most common is **n to 2^n** type of decoder e.g. 2 to 4, 3 to 8 and 4 to 16 line decoders.

Decoders

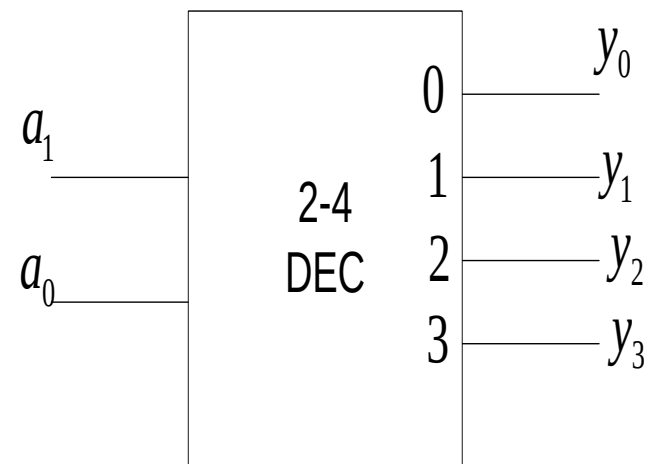
Example:

➤ Show truth table and block diagram of 2 to 4 line decoder

➤ Truth table

Inputs		Outputs			
a_1	a_0	y_0	y_1	y_2	y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

➤ Block diagram



Decoders

- From truth table it can be observed that the output variables are mutually exclusive because only one output can be equal to 1 at any one time.
- The output line whose value is equal to 1 represents the minterm equivalent of the binary number presently available in the input lines.

Decoders

➤ From truth table we can write

$$y_0 = \overline{a_1} \overline{a_0}$$

$$y_1 = \overline{a_1} a_0$$

$$y_2 = a_1 \overline{a_0}$$

$$y_3 = a_1 a_0$$

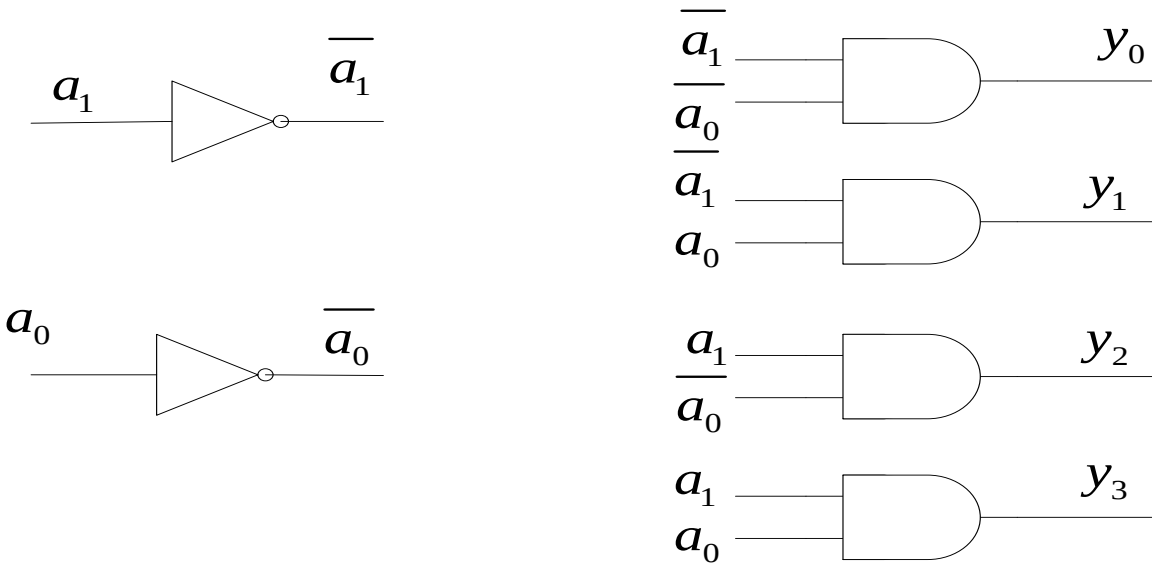


Figure: Implementation of 2 to 4 line decoder

Decoders

Logic design using decoder

- Observe equations below

$$y_0 = \overline{a_1} \overline{a_0} \qquad y_1 = \overline{a_1} a_0$$

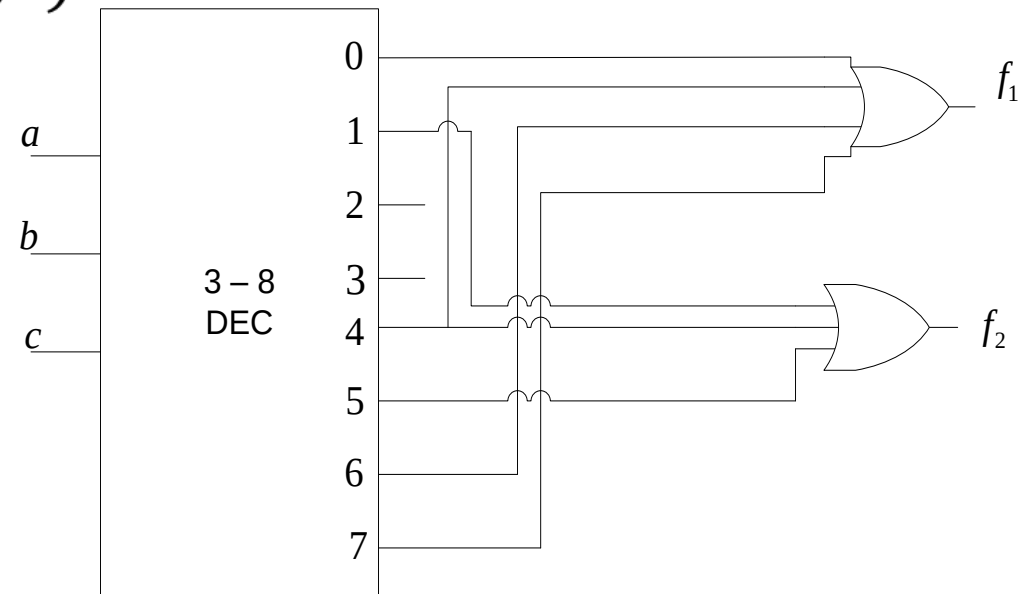
$$y_2 = a_1 \overline{a_0} \qquad y_3 = a_1 a_0$$

- That each output of decoder generate a minterm.
- A 2 to 4 line decoder generates 4 minterms using two variables
- Therefore a n to 2^n line decoder is very convenient to generate n variable minterm and realize a SOP expression

Decoders

Logic design using decoder

- **Example 1:** implement the following function using 3 to 8 decoder
 - $f_1(a, b, c) = \sum m(0, 4, 6, 7)$
 - $f_2(a, b, c) = \sum m(1, 4, 5)$



Decoders

Logic design using decoder

- **Example 2:** implement the following functions expressed in maxterm canonical form in two possible ways using a 3 to 8 decoder.
 - $f_1(a, b, c) = \prod M(2,3,4,5,7)$
 - $f_2(a, b, c) = \prod M(0,2,4,6,7)$

Decoders

Logic design using decoder

- The number of inputs to each or gate can be minimized by implement the f' instead of f and finally invert the OR output to get f .

Example:

- Implement the following functions using a decoder minimizing the number of inputs to be summed.

$$f_1(a, b, c) = \sum m(0, 2, 3, 5, 6, 7)$$

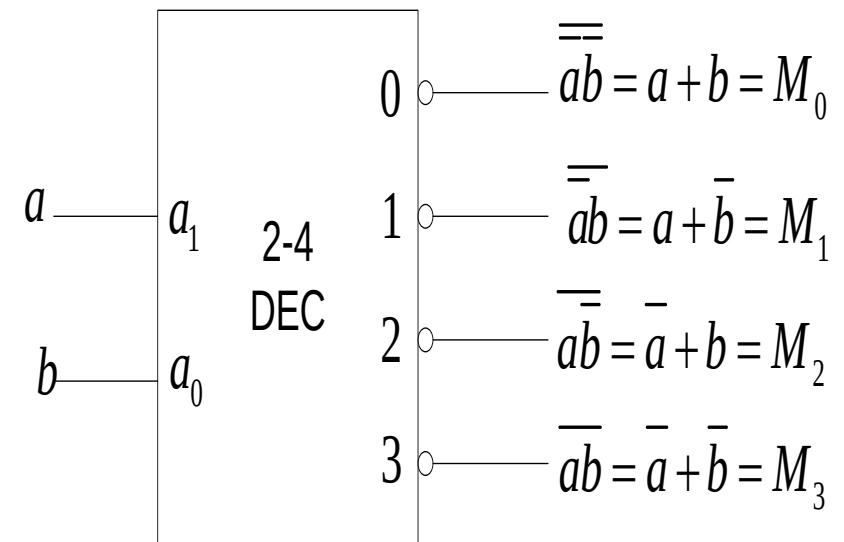
$$f_2(a, b, c) = \sum m(1, 3, 4, 6, 7)$$

Decoders

Logic design using decoder

- The decoder in IC packages usually have an active low output. The truth table and symbol of such 2 to 4 line decoder.
- These generate maxterms at their output and are useful in implementing canonical POS expression.

Inputs		Outputs			
a_1	a_0	y_0	y_1	y_2	y_3
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0



Decoders

Logic design using decoder

- **Example 1:** implement the following functions in maxterm canonical form using 2 to 4 line decoder with active low outputs.
 - $f_1(a, b) = \prod M(1, 3)$
 - $f_2(a, b) = \prod M(0, 2, 3)$
- **Example 2:** implement the following using a 3 to 8 decoder with active low outputs.
 - $f_1(a, b, c) = \sum m(1, 3, 5, 6)$
 - $f_1(a, b, c) = \sum m(0, 2, 5, 6)$

Decoders

Logic design using decoder

- Functions expressed in maxterm canonical forms can also be implemented using decoder.
- **Example:** implement the following functions expressed in maxterm canonical form in two possible ways using a 3 to 8 decoder.

$$f_1(a, b, c) = \prod M(2, 3, 4, 5, 7)$$

$$f_2(a, b, c) = \prod M(0, 2, 4, 6, 7)$$

Decoders

Decoders with Enable Input

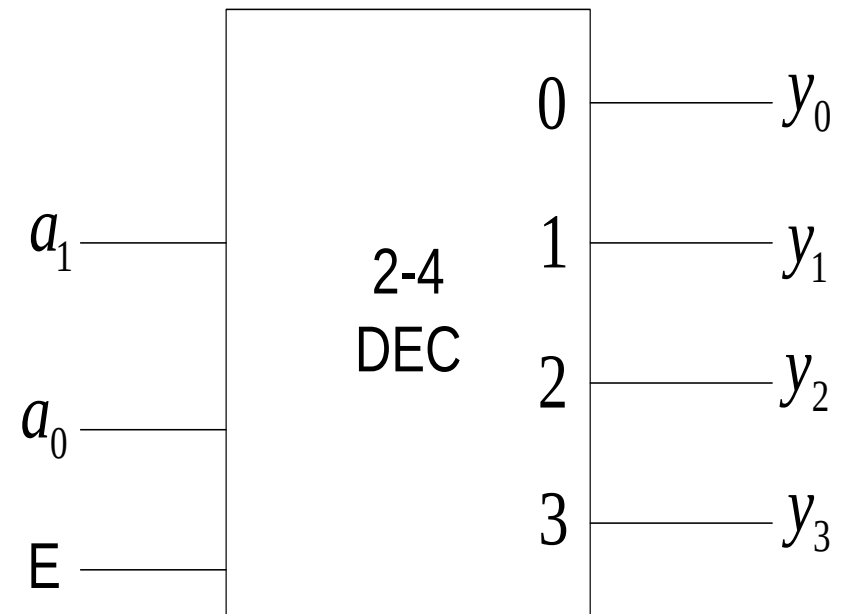
- On previous decoder one of the input is either 1 or 0 for active high and low outputs respectively.
- Enable input is used for some application where more than one output need to be at 0 or 1 state.
- Enable inputs are also useful in cascading decoders to increase their input and outputs lines.

Decoders

Decoders with Enable Input

- The truth table and symbol of 2 to 4 line decoder with enable for uncomplemented (active high) output is as shown

Inputs			Outputs			
E	a_1	a_0	y_0	y_1	y_2	y_3
0	x	x	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

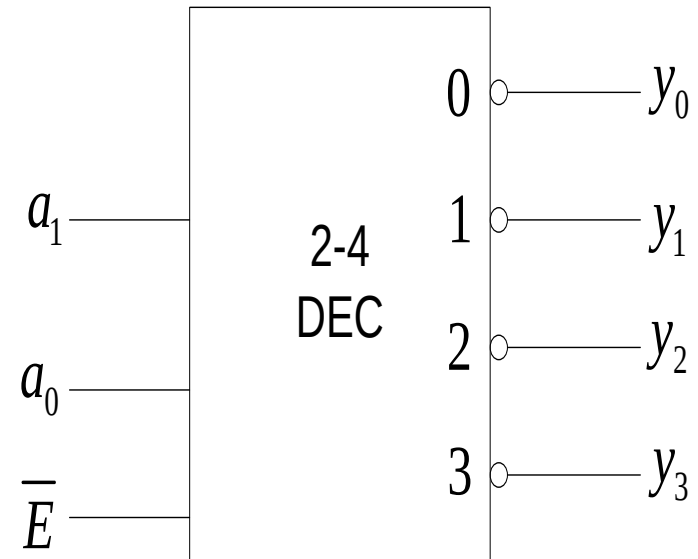


Decoders

Decoders with Enable Input

- The enable input can also be included in decoder with complemented (active low) output as shown

Inputs			Outputs			
E	a_1	a_0	y_0	y_1	y_2	y_3
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0
1	x	x	1	1	1	1



Encoders

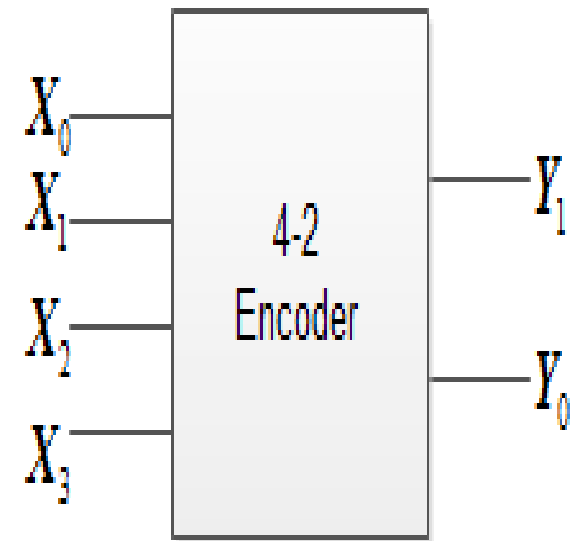
- An encoder has **2^n inputs** and **n outputs**.
- Outputs the binary value of the selected (or active) input.
- Performs the inverse operation of a decoder.
- Some examples of encoders are 4 to 2 line and 8 to 3 line encoders

Encoders

- Truth Table of 4-2line encoder

X_0	X_1	X_2	X_3	Y_1	Y_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

- Block diagram of 4-2line encoder



Priority Encoders

- If more than one input is active the higher order input has priority over the lower order input
- The higher value is encoded at the output
- A valid indicator is included to indicate whether or not the output is valid
- Output is invalid (0) when no inputs are active.
- Output is valid (1) when at least one input is active

Priority Encoders

➤ The truth table of 4-2 line priority encoder

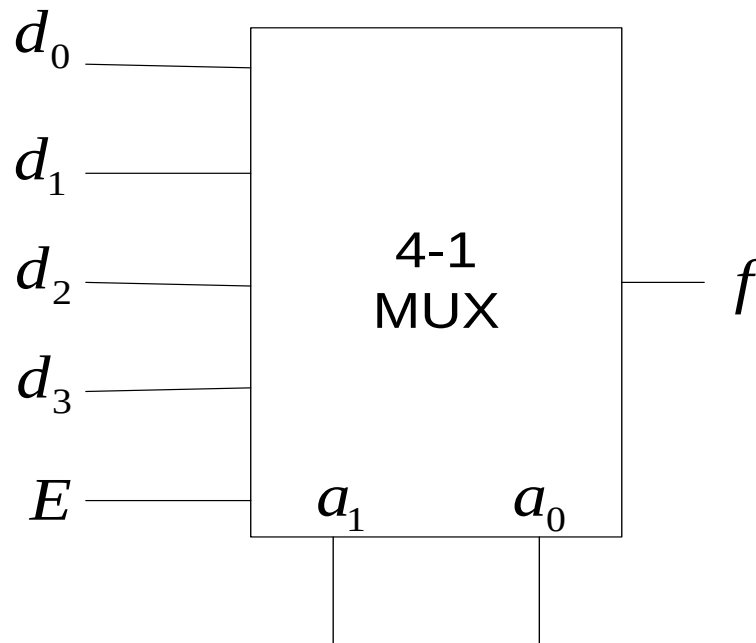
X_0	X_1	X_2	X_3	Y_1	Y_0	Valid
0	0	0	0	0	0	0
1	0	0	0	0	0	1
x	1	0	0	0	1	1
x	x	1	0	1	0	1
x	x	x	1	1	1	1

Multiplexer

- A **multiplexer (MUX)** selects one data line from two or more input lines and routes data from the selected line to the output
- Multiplexer has a set of **n address** lines(select inputs) to select one of these **2^n** inputs.
- Multiplexer also has ***one or more enable inputs***.

Multiplexer

➤ Symbol of 4-1 MUX



Multiplexer

- Truth table of 4 to 1 MUX

Row no.	E	Sel Line		Inputs				Output
		a_1	a_0	d_0	d_1	d_2	d_3	f
0	0	X	X	X	X	X	X	0
1	1	0	0	0	X	X	X	0
2	1	0	0	1	X	X	X	1
3	1	0	1	X	0	X	X	0
4	1	0	1	X	1	X	X	1
5	1	1	0	X	X	0	X	0
6	1	1	0	X	X	1	X	1
7	1	1	1	X	X	X	0	0
8	1	1	1	X	X	X	1	1

Multiplexer

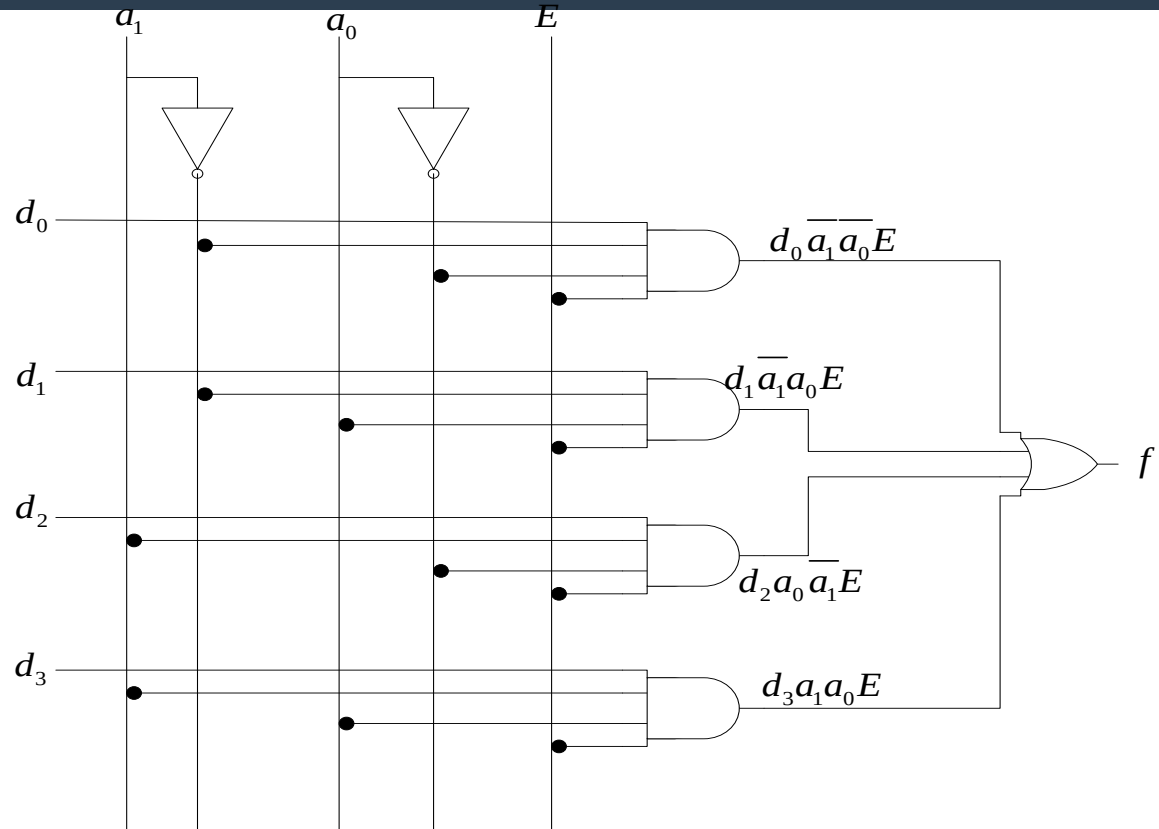
- From truth table functional description can be represented in a functional table.

E	a_1	a_0	f
0	X	X	0
1	0	0	d_0
1	0	1	d_1
1	1	0	d_2
1	1	1	d_3

- The output can be expressed as follows
$$f = (d_0 \overline{a_1} \overline{a_0} + d_1 \overline{a_1} a_0 + d_2 a_1 \overline{a_0} + d_3 a_1 a_0)E$$

Multiplexer

- The 4 to one multiplexer can be implemented as shown in figure



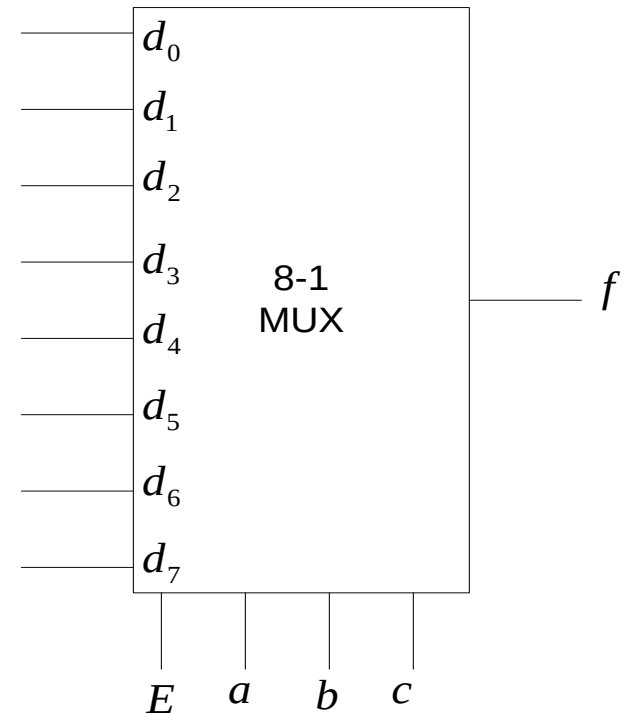
- Several multiplexer are available in IC packages.

Multiplexer

Logic Design using Multiplexers

- MUX is ideal to implement functions expressed as a sum of their minterms.
- Let us look on 8 to 1 MUX

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
0	0	0	d_0
0	0	1	d_1
0	1	0	d_2
0	1	1	d_3
1	0	0	d_4
1	0	1	d_5
1	1	0	d_6
1	1	1	d_7



Multiplexer

Logic Design using Multiplexers

- Let enable $E=1$. equation for a 8 to 1 MUX with reference to figure can be written as

$$f(a, b, c) = d_0 \bar{a} \bar{b} \bar{c} + d_1 \bar{a} \bar{b} c + d_2 \bar{a} b \bar{c} + d_3 \bar{a} b c + d_4 a \bar{b} \bar{c} + d_5 a \bar{b} c + d_6 a b \bar{c} + d_7 a b c$$

$$f(a, b, c) = d_0 m_0 + d_1 m_1 + d_2 m_2 + d_3 m_3 + d_4 m_4 + d_5 m_5 + d_6 m_6 + d_7 m_7$$

Multiplexer

Logic Design using Multiplexers

- To implement a three variable function expressed in terms of its minterms, simply needed to set the data input lines as $d_i = 1$ for $f = 1$ and $d_i = 0$ for $f = 0$

Multiplexer

- **Example 1:** Implement the following function using 8 to 1 MUX.

$$f(a, b, c) = \sum m(0, 4, 5, 6)$$

- **Example 2:** Implement following function using 4 to 1 MUX

$$f(a, b, c) = \sum m(0, 1, 2, 7)$$

- **Example 3:** Implement $f(a, b, c) = \sum m(1, 4, 5, 7)$ using 4 to 1 MUX using b and c as the address lines.

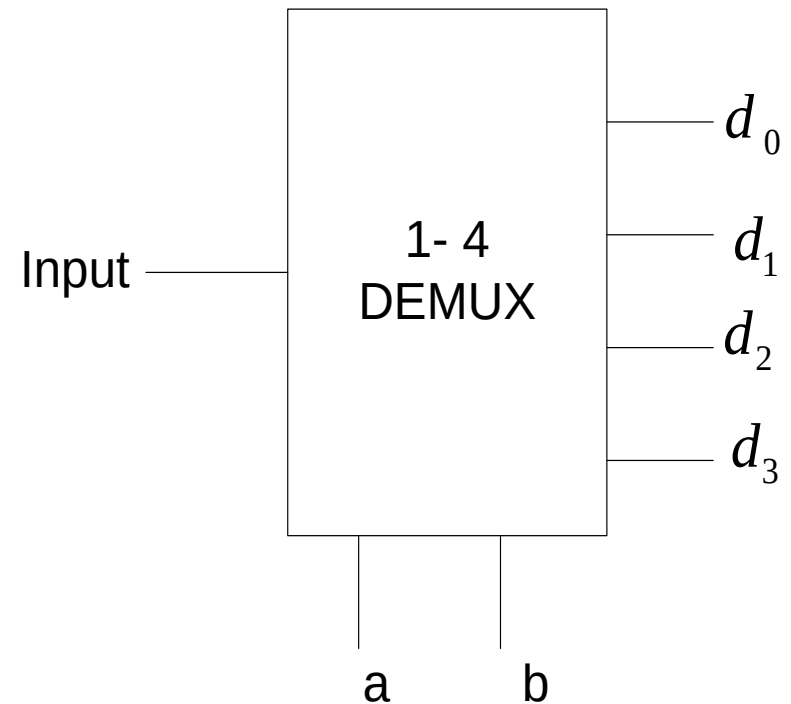
Demultiplexer

- It is the reverse operation of the *multiplexer*.
- A *demultiplexer* has an input line, 2^n output lines and n select lines.
- A *demultiplexer* is the logic circuit that receives information through a single input line and transmits the same information over one of the possible output lines.
- The output line that gets the information present on the input line is decided by the bit status of the selection lines.

Demultiplexer

- Example of 1 to 4 DEMUX

Input t	Select		Output			
	a	b	d_0	d_1	d_2	d_3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1



Tutorials

- Design an excess -3 to 8421 code converter using a 4 to 16 decoder with enable input $\bar{E}$ and construct using NAND gates, so as to minimize the gate inputs.
- Construct 3 to 8 decoder using 2 to 4 line decoder.
- Implement the function $f(a, b, c, d) = \sum m(0,1,5,6,7,9,10,15)$ using 8 to 1 MUX treat a,b and c as the select lines. Implement using a 4 to 1 MUX with a and b as select line

Tutorials

- Implement a full adder circuit with a decoder and two OR gates.

Homework

- Study on
 - Look ahead Carry adder





Next Lecture